emdash — Functorial programming for ω-categories in Lambdapi (v3.2 arrow induction)

https://github.com/hotdocx/emdash

Abstract

We present the v3.2 architecture of **emdash**, a Lambdapi specification for a programming language and proof assistant for strict/lax ω-categories. The main result of this iteration is not a new datatype of paths. It is a category-theoretic arrow-induction principle built from directed families, dependent sums, dependent products, and dependent homs, then checked by normalization in Lambdapi.

The strict/lax distinction is essential. Ordinary functoriality is oriented as strict computation, but transport in directed families may preserve canonical total arrows only up to a displayed transport-comparison cell. The arrow-induction theorem is therefore formulated first as a source-indexed telescope; its Sigma-total presentation is derived only after the relevant transport comparison data have been internalized.

The foundational construction is the directed dependent hom. Given a category-valued family $E : K \to \mathbf{Cat}$, objects $u \in E[x]$ and $v \in E[y]$, and a base arrow $f : x \to y$, the fibre over f is

$$\mathrm{Hom}_{E[y]}(E[f](u), v).$$

This construction is used twice: it describes homs in Sigma totals, and it describes the action of sections over base arrows. From it, v3.2 builds the outgoing-arrow category

$$\mathrm{PathOut}_Z(x) = \Sigma_{y:Z}\, \mathrm{Hom}_Z(x, y)$$

and the canonical arrow

$$\rho_{x,y,p} : (x, \mathrm{id}_x) \to (y, p).$$

The induction principle can be stated directly in the surface language. For a moving source $x : Z$ and a motive E over outgoing arrows from x, there is a functor from the fibre of E at the reflexive outgoing arrow to the category of sections of E:

```
 x :^n Z ;
E :^n (PathOut_Z(x) ⊢ Cat) ;
E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q].
```

It sends $u : E[(x, \mathrm{id}_x)]$ to the section $q \mapsto E[\rho_q](u)$. The Lambdapi implementation packages this telescope as `PathInd_transfd(Z)`.

Its most visible checked consequence is directed transitivity. For $p : x \to y$ and $q : y \to z$, the composition instance of arrow induction normalizes to ordinary categorical composition:

$$\mathrm{path\_comp\_func}(p)[z][q] \rightsquigarrow q \circ p.$$

1. Introduction

Higher category theory is hard to mechanize because coherence is not a small side condition. Every definition carries functoriality, naturality, and higher-dimensional compatibility data. In ordinary mathematical prose, much of this data disappears into phrases such as "by functoriality" or "by naturality." In a proof assistant, those phrases either become explicit proof terms or they must compute.

The emdash design chooses the second option whenever the equation is structural enough to be part of the computational theory. Categories, functors, transformations, category-valued families, dependent sums, dependent products, and dependent homs are internal objects of the Lambdapi specification. Coherence facts are oriented as rewrite and unification rules with stable normal forms.

This paper explains the v3.2 development for a reader who knows dependent type theory or HoTT, but not the emdash codebase. The central claim is:

```
category-theoretic arrow induction can be expressed as a computational theorem.
```

The word "arrow" is important. The theorem is not HoTT identity elimination for paths in a type. It is an induction principle for outgoing directed arrows inside a category. The construction resembles path induction because every outgoing arrow $p : x \to y$ has a canonical comparison from the reflexive outgoing arrow (x, id_x) to (y, p), but that comparison lives in a Sigma category of arrows, not in an identity type.

1.1 What v3.2 Contributes

The v3.2 article focuses on five checked contributions.

1. **Directed dependent homs.** Homs in total categories and section actions are organized by one dependent-hom construction over a base arrow.
2. **Outgoing-arrow categories.** For each object $x : Z$, $\mathrm{PathOut}_Z(x)$ is the Sigma total of the representable family $\mathrm{Hom}_Z(x, -)$.
3. **Synthetic arrow induction.** For `x :^n Z ; E :^n (PathOut_Z(x) ⊢ Cat)`, the checked construction has type `E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q]`; the kernel packages the source-indexed telescope as `PathInd_transfd(Z)`.
4. **Strict and lax directed transport.** General displayed functors carry computable transport-comparison cells over base arrows; strict or cartesian constructors, including representable precomposition, collapse those cells to canonical identities.
5. **Computational composition.** The transitivity benchmark reduces a fully expanded path-induction expression to $q \circ p$, represented in the kernel by the normal form `comp_fapp0 Z x y z q p`.

The rest of the paper builds these facts in the order a reader needs them: first the foundation, then dependent homs, then outgoing arrows, then the induction theorem, and finally the composition computation.

1.2 What "Checked" Means

The paper uses checked in the Lambdapi sense. An assertion

```
Γ ⊢ left ≡ right
```

is accepted only when `left` and `right` are convertible under beta-reduction, definitions, rewrite rules, and unification rules. For the arrow-induction story, the checked evidence has three layers.

1. The theorem interfaces typecheck, including the unfolded telescope `E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q]`.
2. Their projections compute; for example, the source-indexed theorem reduces at source `x` to the fixed-source construction introduced in Section 5.
3. Fully expanded applications normalize to ordinary categorical terms, for example the composition benchmark reduces to $q \circ p$, whose kernel normal form is `comp_fapp0 Z x y z q p`.

Thus the paper is not claiming only that the theorem is mathematically plausible. The theorem surface, its projections, and the benchmark computation are part of the same executable rewrite system.

We follow a formula-first convention for implementation names. When the paper mentions a Lambdapi identifier such as `Sigma_cat` or `PathInd_transfd`, the surrounding display gives the corresponding mathematical or emdash surface notation first; Appendix A collects the identifier correspondences.

2. A Type-Theoretic Foundation For Directed Families

The surface notation is intentionally close to dependent type theory, but the base of a family is directed. A category K has real arrows, not just points. Consequently, a family over K must explain how fibre objects move along base arrows.

The core reading is:

```
 A : Cat                  a category
x : A                     an object, formally x : Obj(A)
a →^A b                   hom-category Hom_A(a,b)
F : A ⊢ B                 ordinary functor
F[x]                      object action
F[f]                      arrow action
E : K ⊢ Cat               category-valued family over K
E[k]                      fibre category at k
E[f](u)                   transport of u along f : k → k'
```

The hom between two objects is itself a category. A 1-cell $f : x \to y$ is an object of $\mathrm{Hom}_A(x, y)$; a 2-cell is an arrow in that hom-category; higher cells are obtained by iterating the same pattern. This is the ω-oriented part of the foundation.

The universe of categories is also a category:

```
 Obj(Cat)      = categories
Hom_Cat(A,B)  = A ⊢ B
```

So a category-valued family is just a functor into `Cat`. The implementation calls the category of such families `Catd(K)`, but the mathematical reading is:

```
E : K ⊢ Cat.
```

A morphism of families is a natural family of functors:

```
FF : k :^n K ; E[k] ⊢ D[k]
FF[k] : E[k] ⊢ D[k].
```

A transformation between such family morphisms is a family of transformations natural in the same directed base:

```
ϵ : FF ⇒ GG
ϵ[k] : FF[k] ⇒ GG[k].
```

The notation `:^n` marks an ordinary directed index in the current surface syntax. The paper uses typographic operators such as `→`, `→_`, `⇒`, and `⇒_`; the parser-oriented ASCII spellings are recorded in the syntax report. Mixed variance appears on the family occurrence, for example `A[z^-] ⊢_[z] B[z]`, not by introducing older binder modes.

3. Dependent Hom, Sigma Totals, And Sections

The key construction for v3.2 is the dependent hom over a base arrow. Let

```
E : K ⊢ Cat
x y : K
u : E[x]
v : E[y].
```

For every base arrow $f : x \to y$, transport sends u to $E[f](u)$ in the fibre over y. The dependent hom over f is then:

$$\mathrm{Hom}_{E[y]}(E[f](u), v).$$

As a whole, this is not only a pointwise formula. It is a Cat-valued functor over the base hom:

```
homd_E(x,u,y,v) : Hom_K(x,y)^op ⊢ Cat
homd_E(x,u,y,v)[f] = Hom_{E[y]}(E[f](u),v).
```

The Lambdapi identifier for this dependent-hom functor is `homd_`. The opposite on the base hom is an implementation convention that makes the functorial action line up with the current normal forms. The reader-facing content is simply: a dependent arrow from (x, u) to (y, v) consists of a base arrow and a fibre arrow above it.

3.1 Sigma Totals

The dependent sum of a directed family is a total category:

```
Σ_k E[k].
```

Its objects are pairs:

```
(k,u)       with k : K and u : E[k].
```

The hom category in a total category is organized by the dependent hom. At the level of 1-cell data it has the familiar dependent-pair shape:

```
Hom_{ΣE}((x,u),(y,v))
  = Σ f : x →^K y, Hom_{E[y]}(E[f](u),v).
```

The kernel keeps the opposite orientation needed for higher hom-action explicit, so the checked normal form is an opposite of a Sigma category over the opposite base hom. This distinction affects higher cells and normal forms, not the ordinary description of a total arrow.

Equivalently, an arrow in the total category is a pair:

```
 (f, α)
f : x →^K y
α : E[f](u) →^{E[y]} v.
```

The implementation identifier for $\Sigma_k E[k]$ is `Sigma_cat(E)`; `sigma_arrow(E,p,α)` names the pair `(p,α)`. The canonical transported total arrow is `sigma_transport_arrow(E,p,u)`:

```
sigma_transport_arrow(E,p,u) : (x,u) → (y,E[p](u))
```

whose fibre component is the identity at the transported endpoint.

3.2 Pi Sections

The dependent product is the category of sections:

```
Π_k E[k].
```

An object $s : \Pi_k E[k]$ assigns a fibre object $s[k] : E[k]$ for every $k : K$, and it also carries action over base arrows:

```
s[f] : Hom_{E[y]}(E[f](s[x]), s[y])
```

for $f : x \to y$. This is the same dependent-hom shape as the Sigma hom, specialized to the section's endpoints. In v3.2 this shared architecture is important: total-category arrows and section actions are not independent features with unrelated computation rules.

The implementation name for the section category is `Pi_cat(E)`. In the surface notation the article writes:

```
Π (k :^n K), E[k].
```

Section evaluation is written $s[k]$ in the surface notation; its Lambdapi projection head is `piapp0(s,k)`.

4. Outgoing Arrows And The Canonical `rho` Arrow

Now fix a category Z and an object $x : Z$. The represented family at x is:

```
Rep_Z(x)[y] = Hom_Z(x,y).
```

Its action along a base arrow $p : x \to y$ is representable precomposition:

```
 Rep_transport(p) : Rep_Z(y) ⊢ Rep_Z(x)
Rep_transport(p)[z][q] = q ∘ p.
```

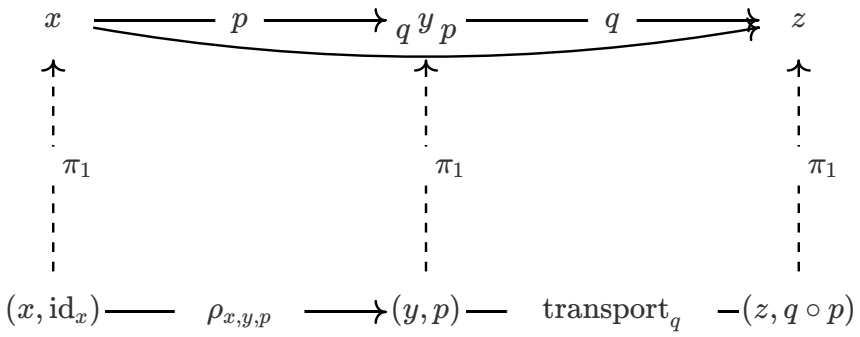
The outgoing-arrow category from x is the Sigma total of this representable:

```
PathOut_Z(x) = Σ y :^n Z, Rep_Z(x)[y]
             = Σ y :^n Z, Hom_Z(x,y).
```

An object of `PathOut_Z(x)` is a pair `(y,p)` with `p : x →^Z y`. The distinguished reflexive outgoing arrow is:

```
reflout_x = (x,id_x).
```

The first diagram shows `PathOut_Z(x)` as a total category over Z.



For every object `(y,p)` of `PathOut_Z(x)`, there is a canonical arrow:

```
rho_{x,y,p} : reflout_x →^PathOut_Z(x) (y,p).
```

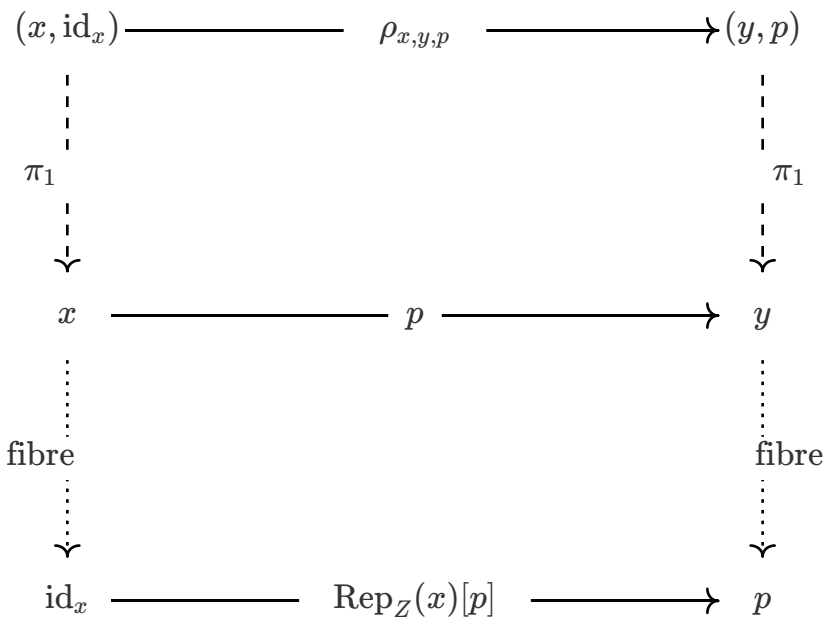
This is not postulated. It is the Sigma transport arrow of the representable family:

```
rho_{x,y,p} = sigma_transport_arrow(Rep_Z(x), p, id_x).
```

The endpoint computation is:

```
Rep_Z(x)[p](id_x) = p.
```

Thus the fibre component of `rho` is the identity at the computed endpoint p.



The `rho` arrows assemble into a section:

```
pathout_refl_arrow_sec(x)
  : Π q :^n PathOut_Z(x),
      reflout_x →^PathOut_Z(x) q

pathout_refl_arrow_sec(x)[(y,p)] = rho_{x,y,p}.
```

This section is the categorical reason an induction principle is available: every outgoing arrow has a canonical arrow from the reflexive outgoing arrow.

5. Arrow Induction

Let `E : PathOut_Z(x) ⊢ Cat` be a motive over outgoing arrows, and let

```
u : E[reflout_x].
```

Fixed-source arrow induction constructs a section:

```
path_ind_sec(Z,x,E,u) : Π q :^n PathOut_Z(x), E[q].
```

Its computation at an outgoing arrow `(y,p)` is:

```
path_ind_sec(Z,x,E,u)[(y,p)] = E[rho_{x,y,p}](u).
```

In words: to define an element of the motive at every outgoing arrow, it suffices to define it at the reflexive outgoing arrow and then transport along the canonical `rho` arrow.

The checked internal version of the same statement uses ordinary fibre transport in the family `E`. The kernel head `fib_cov_tapp0_func` names the functor $f \mapsto E[f](u)$:

```
piapp0(path_ind_sec(Z,x,E,u),(y,p))
  = fib_cov_tapp0_func(PathOut_Z(x),E,reflout_x,(y,p),u)(rho_{x,y,p}).
```

This fixed-source construction is packaged as a shaped functor:

```
PathInd_func(Z,x)
  : E :^n (PathOut_Z(x) ⊢ Cat) ;
      E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q]

PathInd_func(Z,x)[E](u) = path_ind_sec(Z,x,E,u).
```

5.1 The Telescope Theorem

The primary v3.2 theorem lets the source object vary. In unfolded surface notation, the theorem is the following source-indexed construction:

```
 x :^n Z ;
E :^n (PathOut_Z(x) ⊢ Cat) ;
E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q].
```

Its value on $u : E[(x, \mathrm{id}_x)]$ is the section whose component at `(y,p)` is `E[rho_{x,y,p}](u)`.

The kernel packages this theorem as a transformation between two source-indexed constructions. For each $x : Z$, the motive category is:

```
PathOutMotives_Z[x] = PathOut_Z(x) ⊢ Cat.
```

The source and target constructions are:

```
 source[x,E] = E[(x,id_x)]
target[x,E] = Π q :^n PathOut_Z(x), E[q].
```

The implementation names for these are `PathOutReflEval_Z` and `PathOutPi_Z`, and the packaged theorem is:

```
PathInd_transfd(Z)
  : x :^n Z ; PathOutReflEval_Z[x] ⇒ PathOutPi_Z[x].
```

This packaging is important to the implementation because naturality in the moving source object is part of the theorem's type, not a separate external square.

Along a base arrow $p : x \to y$, `PathOut` is contravariant in the source. Precomposition gives a functor

```
 PathOut_Z(p) : PathOut_Z(y) ⊢ PathOut_Z(x)
PathOut_Z(p)[(z,q : y → z)] = (z,q ∘ p).
```

A motive E over `PathOut_Z(x)` is transported by pullback along this action:

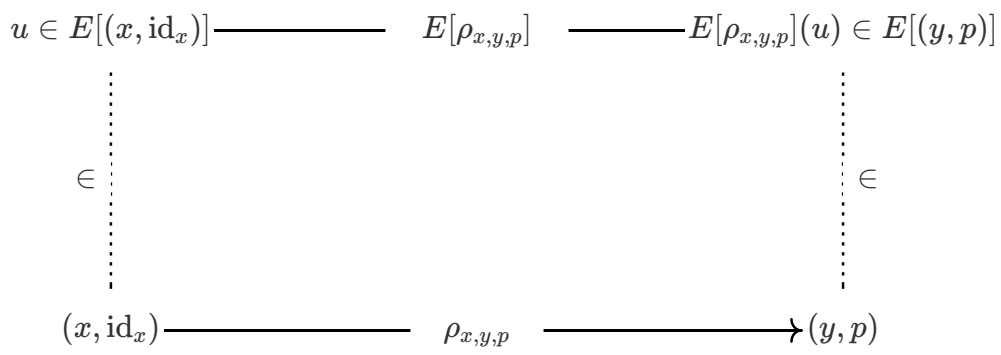
```
p_*E = PathOut_Z(p)^*E : PathOut_Z(y) ⊢ Cat.
```

In Lambdapi this is the expression `Pullback_catd(E, PathOut_transport_func(p))`. The source-side transport is concrete:

```
PathIndSrc_transport(p,E) = E[rho_{x,y,p}]
  : E[(x,id_x)] ⊢ E[(y,p)].
```

The target-side transport is section pullback:

```
PathIndTgt_transport(p,E)
  : Π q :^n PathOut_Z(x), E[q]
    ⊢ Π q :^n PathOut_Z(y), E[PathOut_transport(p)[q]].
```



5.2 The Derived Sigma-Total Presentation

There is also an ordinary total-category presentation of the theorem:

```
PathInd_funcd(Z)
 : (x,E) :^n Σ x :^n Z, (PathOut_Z(x) ⊢ Cat) ;
     E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q].
```

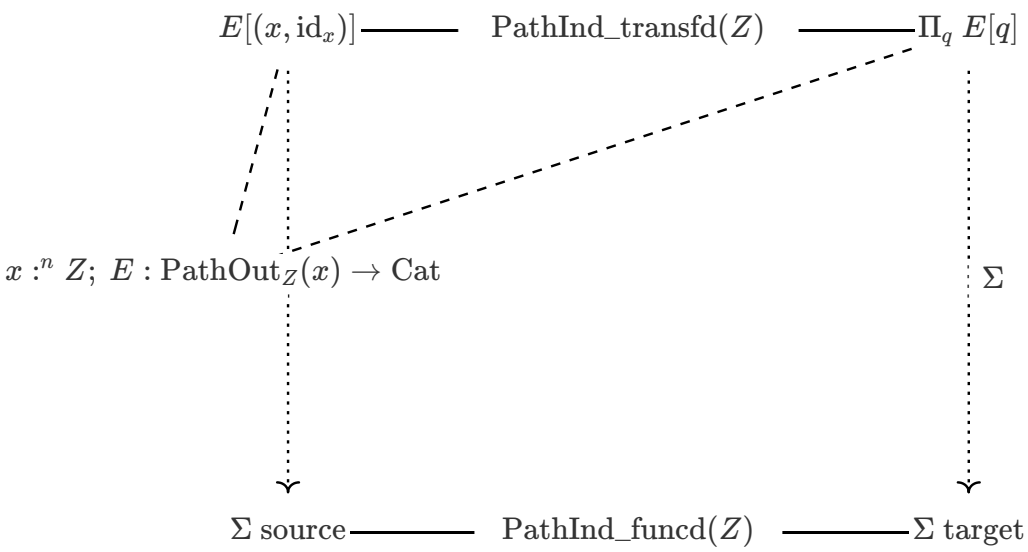
It is derived, not primitive:

```
PathInd_funcd(Z) = Sigma_transfd_funcd(PathInd_transfd(Z)).
```

Equivalently, it is the Sigma-totalization of the telescope theorem. Its component at `(x,E)` reduces back to the object component of the fixed-source functor:

```
PathInd_funcd(Z)[(x,E)] = path_ind_func_fapp0(Z,x,E).
```

This distinction is important. The theorem is naturally telescope-shaped; the Sigma-total form is a useful presentation for ordinary functor-level applications and expanded regression checks.



6. Strictness, Laxity, And Directed Induction

In ordinary dependent type theory and HoTT, substitution is organized around variables and identity/path structure. The computation rules for identity elimination are strict at reflexivity, and many uses of transport are judgemental only after the relevant path has reduced to `refl`.

In emdash v3.2 the base of a family is a directed category. A family $E : K \to \mathbf{Cat}$ transports fibre objects along actual arrows $p : x \to y$:

```
E[p](u) : E[y].
```

This directed transport is strict for ordinary functoriality: functor actions are oriented so that composites compute as composites. However, a morphism of families

```
FF : k :^n K ; E[k] ⊢ D[k]
```

does not in general preserve canonical transport arrows strictly. Over $p : x \to y$ and $u : E[x]$, the two evident endpoints are

```
 D[p](FF[x](u))
FF[y](E[p](u)).
```

A displayed functor therefore has a directed laxity phenomenon at the component level. We write the associated transport-comparison component as `χ^{FF}_{p,u}` (read as `cmp(FF,p,u)`), reserving `λ` for lambda abstraction:

```
χ^{FF}_{p,u} : D[p](FF[x](u)) →^{D[y]} FF[y](E[p](u)).
```

This is useful mathematical notation, but it is not a primitive operation in the active kernel. The computational owner is the internal displayed hom-action. Its capped projection has the form

```
fdapp1_int_hom_fapp0(FF,p,u,α)
  : D[p](FF[x](u)) →^{D[y]} FF[y](v)
```

for a fibre arrow $\alpha : E[p](u) \to v$. Thus the Sigma-map action on a total arrow is represented as

```
Σ(FF)(p,α) = (p, fdapp1_int_hom_fapp0(FF,p,u,α)).
```

The familiar composite $FF[y][\alpha] \circ \chi^{FF}_{p,u}$ is only a surface reading of this capped projection. The article does not take it as the definition, because the active v3.2 design deliberately avoids reconstructing Sigma maps from a separate whole-comparison operation.

The canonical identity case extracts the component-level cell:

```
fdapp1_int_hom_fapp0(FF,p,u,id_{E[p](u)})
  = fdapp1_int_cell(FF,p,u).
```

This is the sense in which laxity becomes visible: it is observed through the internal displayed hom-action and its Sigma-map fibre component, not supplied as an independent external naturality square.

Strict or cartesian constructors add focused computation rules making the comparison cell collapse. Representable precomposition is one such strict case: for $p : x \to y$, the family morphism

```
Rep_transport(p) : Rep_Z(y) ⊢ Rep_Z(x)
```

has cartesian behaviour on the canonical identity fibre arrow. Concretely, for $q : y \to z$, the comparison component reduces to the identity at the composite:

```
χ^{Rep_transport(p)}_{q,id_y} = id_{q ∘ p}.
```

This is why the `PathOut` and composition benchmarks compute strictly even though the surrounding directed-family theory permits lax displayed transport.

The formulation of arrow induction follows this discipline. The primary theorem is the source-indexed telescope:

```
 x :^n Z ;
E :^n (PathOut_Z(x) ⊢ Cat) ;
E[(x,id_x)] ⊢ Π (q :^n PathOut_Z(x)), E[q].
```

Moving the source object along $p : x \to y$ transports motives by pullback along `PathOut_Z(p)`. The source side is the concrete map $E[\rho_{x,y,p}]$, and the target side is section pullback. The derived Sigma-total theorem is used to inspect this structure at canonical transported objects. It should not be read as a claim that arbitrary non-cartesian Sigma-total arrows preserve the induction structure strictly.

7. Composition As The Main Computation

The most concrete application of arrow induction is directed transitivity. Fix $x : Z$. Define the composition target family:

```
CompTarget_Z(x)[y] = Rep_Z(y) ⊢ Rep_Z(x).
```

Pull it back along the projection `PathOut_Z(x) → Z`:

```
CompMotive_Z(x)[(y,p)] = Rep_Z(y) ⊢ Rep_Z(x).
```

Path induction at the identity displayed functor gives:

```
path_comp_sec(x)
 = path_ind_sec(CompMotive_Z(x), id_Rep_Z(x)).
```

Its first application to $p : x \to y$ is a functor:

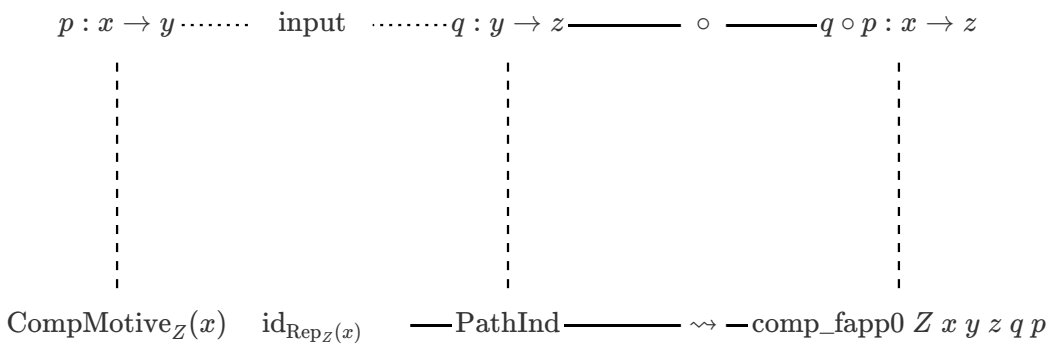
```
path_comp_func(p) : Rep_Z(y) ⊢ Rep_Z(x).
```

Its action on $q : y \to z$ computes to ordinary composition:

```
path_comp_func(p)[z][q] = q ∘ p.
```

The kernel normal form for this composite is:

```
comp_fapp0 Z x y z q p.
```



Both theorem presentations reach this normal form. The primary telescope route is checked as:

```
PathInd_transfd(Z)[x][CompMotive_Z(x)](id_Rep_Z(x))[p][z][q]
 = comp_fapp0 Z x y z q p.
```

The derived Sigma-total route is checked as:

```
PathInd_funcd(Z)[(x,CompMotive_Z(x))](id_Rep_Z(x))[p][z][q]
 = comp_fapp0 Z x y z q p.
```

The computation is not a special-purpose rewrite for transitivity. It is the ordinary action of the representable family, reached by applying the general arrow-induction theorem to the composition motive.

8. Surface Syntax And Kernel Names

The paper uses the current v3.2 surface syntax. The syntax distinguishes ordinary homs, indexed homs, ordinary functor categories, shaped functor categories, section categories, and displayed transformation categories.

Surface form	Kernel meaning	Role
`a →^C b`	`Hom_cat C a b`	ordinary hom with explicit ambient category
`a → b`	`Hom_cat C a b`	ordinary hom when `C` is clear
`aa[z^-] →_[z]^R bb[z]`	`Hom_catd R aa bb`	indexed/displayed hom
`A ⊢ B`	`Functor_cat A B`	ordinary functor/program category
`z :^n Z ; E[z] ⊢ D[z]`	`Functord_cat E D`	shaped functor/program category
`Π (z :^n Z), D[z]`	`Pi_cat D`	terminal-shape section category
`A[z^-] ⊢_[z] B[z]`	`Functor_catd A B`	mixed-variance displayed functor family
`F ⇒ G`	`Transf_cat F G`	ordinary transformation category
`FF[z^-] ⇒_[z] GG[z]`	`Transf_catd A B FF GG`	indexed/displayed transformation category

Two notation choices prevent ambiguity.

First, ordinary ambient categories use superscripts:

```
a →^C b.
```

The operator `→_` is reserved for indexed or displayed homs:

```
aa[z^-] →_[z]^R bb[z].
```

Second, the shaped category

```
z :^n Z ; E[z] ⊢ D[z]
```

does not bind an object variable of `E[z]`. The shape `E[z]` is part of the generalized quantification. If the target depends on an actual object of `E[z]`, the construction is a Sigma-style telescope.

The nested telescope stress test is:

```
k :^n K ; C[k] ⊢ (z :^n Z ; E[k^-;z] ⊢ D[k;z]).
```

This is telescope-style notation, not product-base notation. The order `k;z` is meaningful.

9. Computational Method And Checked Evidence

The formalization relies on a small normalization discipline. Constructions whose projections are used by later theorems are assigned stable semantic owners; readable helper names are routed through those owners rather than duplicating their definitions.

For example:

```
CompTarget_Z(x)[y] = Rep_Z(y) ⊢ Rep_Z(x).
```

The helper for its action on a base arrow routes through `CompTarget_Z(x)`; it is not a second definition of the same operation.

The stable heads that appear in the evidence have direct mathematical readings:

```
 fapp0         ordinary functor object action
fapp1_fapp0  ordinary functor arrow action
tapp0_fapp0  transformation component
tdapp0_fapp0 displayed-transformation component
piapp0        section evaluation
```

Rules are installed at the lowest stable head that carries the relevant discriminator. This is why diagnostic assertions often use canonical forms such as `Hom_cat`, `Functor_cat`, and `Functord_cat` rather than readability wrappers.

The formal artifact separates definitions from executable claims: `emdash3_2.lp` contains the definitions and rewrite rules, while `emdash3_2_checks.lp` contains the conversion assertions used as regression evidence.

Representative checked claims:

Surface	Representative checked claim
Representable	`Rep_Z(x)[y] = Hom_Z(x,y)`
PathOut object layer	`PathOut_Z(x) = Sigma_cat Z (Rep_Z(x))`
PathOut transport	`PathOut_Z(p) : PathOut_Z(y) ⊢ PathOut_Z(x)` and `PathOut_Z(p)[(z,q : y → z)] = (z,q ∘ p)`
Rho construction	`rho_{x,y,p} = sigma_transport_arrow(Rep_Z(x),p,id_x)`
Fixed-source induction	`PathInd_func(Z,x)[E](u) = path_ind_sec(Z,x,E,u)`
Telescope theorem	`PathInd_transfd(Z)[x] = PathInd_func(Z,x)`
Sigma-total theorem	`PathInd_funcd(Z)[(x,E)] = path_ind_func_fapp0(Z,x,E)`
Composition benchmark	`path_comp_func(p)[z][q] = q ∘ p`

10. Supporting Examples, Limitations, And Future Work

The path-induction theorem is the central v3.2 result, but the same normalization discipline appears elsewhere in the file.

Product-valued functors and product projections compute through stable projection functors. Semantic curry and uncurry are present at object level, with capped hom-action checks documenting the current behavior. The remaining higher action for some whole-functor semantic uncurry statements is deferred.

Ordinary adjunctions provide another compact example. The v3.2 interface treats an adjunction as a first-class object:

```
 J : Adjunction(R,L)
left_adj_func(J)     : R ⊢ L
right_adj_func(J)    : L ⊢ R
unit_adj_transf(J)   : id_R ⇒ right(J) ∘ left(J)
counit_adj_transf(J) : left(J) ∘ right(J) ⇒ id_L
```

The component-level triangle reductions are oriented as computation:

```
 counit[f] ∘ left(unit[g]) → f ∘ left(g)
right(counit[g]) ∘ unit[f] → right(g) ∘ f.
```

These are supporting examples, not the lead theorem.

Several parts of the intended language remain outside the present theorem.

- Structural functor logic, including exchange, weakening, and contraction for nested shaped categories, is planned but not implemented.
- General dependent adjunctions `Σ_F ⊣ F^* ⊣ Π_F` remain future work.
- Arbitrary Sigma-map transport is not claimed to be strict without the right strict or cartesian specialization.
- Whole-transformation displayed laxity is deferred; current rules expose the component-level infrastructure needed by the checked examples.
- A presentation facade such as `section_total(s) : K ⊢ Σ_K E` is not yet a named surface constructor.

11. Formal Artifact And Validation

The artifact consists of a Lambdapi development, a companion regression module, and a reproducible paper-rendering pipeline. The active v3.2 sources are:

- `emdash3_2.lp` : definitions, interfaces, and rewrite rules.
- `emdash3_2_checks.lp` : executable assertions and regression checks.
- `reports/EMDASH_FOUNDATIONS.md` : mathematician-facing foundation guide.
- `reports/REPORT_EMDASH_V3_2_CANONICAL_SURFACE_SYNTAX_2026-06-05.md` : notation authority.
- `reports/REPORT_EMDASH_V3_2_CURRENT_STATUS_AND_SOP_2026-05-26.md` : rewrite/debugging SOP.

The claims in this article are validated by the following checks:

```
timeout 60s lambdapi check -w emdash3_2.lp
timeout 60s lambdapi check -w emdash3_2_checks.lp
EMDASH_TYPECHECK_TIMEOUT=60s make check
npm run check:render
```

The first three commands check the formal development and its conversion assertions. The final command validates embedded diagram specifications, builds the renderer, and runs a browser smoke test for the paper artifact.

12. Conclusion

The v3.2 development shows that directed arrow induction can be expressed as computational categorical structure. The foundation is a directed dependent type theory of category-valued families, Sigma totals, Pi sections, and dependent homs. The outgoing-arrow category is a Sigma total of a representable; the canonical `rho` arrow is Sigma transport; and the primary theorem is a source-indexed telescope from the reflexive fibre of a motive to its section category.

The composition benchmark is the visible payoff. Applying arrow induction to the composition motive normalizes to ordinary categorical composition. That is the emdash design goal in miniature: coherence is not merely stated; it computes.

Appendix A. Identifier Glossary

Mathematical notation	Kernel identifier
category of categories	`Cat_cat`
`E : K ⊢ Cat`	`Catd_cat K`
fibre `E[k]`	`Fibre_cat K E k`
transport `E[p]`	`catd_transport_func K E x y p`
`A ⊢ B`	`Functor_cat A B`
`z :^n Z ; E[z] ⊢ D[z]`	`Functord_cat Z E D`
`Π (k :^n K), E[k]`	`Pi_cat K E`
section evaluation `s[k]`	`piapp0 K E s k`
`Σ (k :^n K), E[k]`	`Sigma_cat K E`
total arrow `(p,α)` in `ΣE`	`sigma_arrow K E x y u v p α`
canonical total transport	`sigma_transport_arrow K E x y p u`
Sigma map `Σ(FF)`	`sigma_map_func K E D FF`
dependent hom `homd_E(x,u,y,v)`	`homd_ K E E (id_funcd K E) x u y v`
motive pullback `F^*E`	`Pullback_catd A B E F`
uncurrying a displayed theorem over Sigma	`Sigma_transfd_funcd K R S T η`
`Rep_Z(x)`	`Rep_catd Z x`
`Rep_transport(p)`	`Rep_transport_func Z x y p`
`PathOut_Z(x)`	`PathOut_cat Z x`
`PathOut_Z(p)`	`PathOut_transport_func Z x y p`
`(y,p)` in `PathOut_Z(x)`	`pathout_obj Z x y p`
`(x,id_x)`	`pathout_refl_obj Z x`
`rho_{x,y,p}`	`pathout_refl_arrow Z x y p`
section of all `rho` arrows	`pathout_refl_arrow_sec Z x`
`E[rho_{x,y,p}]`	`pathout_refl_eval_base_func Z x y p E`
transported motive `p_*E`	`pathout_motive_transport_obj Z x y p E`
`PathOutMotives_Z`	`PathOutMotives_catd Z`

`PathOutReflEval_Z`	`PathOutReflEval_funcd Z`
`PathOutPi_Z`	`PathOutPi_funcd Z`
fixed-source induction source family	`PathInd_src_catd Z x`
fixed-source induction target family	`PathInd_tgt_catd Z x`
fixed-source component functor	`path_ind_func_fapp0 Z x E`
fixed-source path induction	`PathInd_func Z x`
telescope path induction	`PathInd_transfd Z`
Sigma-total source family	`PathIndSrc_catd Z`
Sigma-total target family	`PathIndTgt_catd Z`
Sigma-total source transport	`PathIndSrc_transport_func Z x y p E`
Sigma-total target transport	`PathIndTgt_transport_func Z x y p E`
Sigma-total path induction	`PathInd_funcd Z`
composition target	`CompTarget_catd Z x`
action of composition target	`CompTarget_fapp1_func Z x a b p`
composition motive	`CompMotive_catd Z x`
composition section	`path_comp_sec Z x`
composition functor for `p`	`path_comp_func Z x y p`
fibre transport action `E[f](u)`	`fib_cov_tapp0_func K E x y u`
covariant fibre transport	`fib_cov_transf Z D x u`
Sigma-map fibre component	`fdapp1_int_hom_fapp0 K E D FF x y p u v α`
displayed transport-comparison component `χ^{FF}_{p,u}`	`fdapp1_int_cell K E D FF x y p u`

Appendix B. Selected Checked Normal Forms

The article quotes checked normal forms in compact mathematical notation and keeps the fully expanded Lambdapi terms available as regression references. The following statements are covered by `emdash3_2_checks.lp`.

Core PathOut and rho checks:

```
 Rep_Z(x)[y] = Hom_Z(x,y)
PathOut_Z(x) = Sigma_cat Z (Rep_Z(x))
PathOut_Z(p) : PathOut_Z(y) ⊢ PathOut_Z(x)
PathOut_transport(p)[(z,q : y → z)] = (z,q ∘ p)
PathOut_transport(p)[(y,id_y)] = (y,p)
Rep_Z(x)[p](id_x) = p
rho_{x,y,p} = sigma_transport_arrow(Rep_Z(x),p,id_x)
pathout_refl_arrow_sec(x)[(y,p)] = rho_{x,y,p}
PathOut_transport(p)(rho_{y,z,q}) ; rho_{x,y,p}
  = rho_{x,z,q ∘ p}
```

Path-induction projection checks:

```
 PathInd_func(Z,x)[E] = path_ind_func_fapp0(Z,x,E)
PathInd_func(Z,x)[E](u) = path_ind_sec(Z,x,E,u)
path_ind_sec(Z,x,E,u)[(y,p)] = E[rho_{x,y,p}](u)
PathInd_transfd(Z)[x] = PathInd_func(Z,x)
PathInd_transfd(Z)[x][E](u) = path_ind_sec(Z,x,E,u)
PathInd_funcd(Z)[(x,E)] = path_ind_func_fapp0(Z,x,E)
PathInd_funcd(Z)[(x,E)](u) = path_ind_sec(Z,x,E,u)
```

Composition benchmark checks:

```
 CompTarget_Z(x)[y] = Rep_Z(y) ⊢ Rep_Z(x)
CompMotive_Z(x)[(y,p)] = Rep_Z(y) ⊢ Rep_Z(x)
Pi(PathOut_Z(x), CompMotive_Z(x))
  = z :^n Z ; Rep_Z(x)[z] ⊢ CompTarget_Z(x)[z]

path_ind_sec(Sigma_proj1_pullback(D),u) = fib_cov_transf(D,x,u)
path_ind_sec(CompMotive_Z(x), id_Rep_Z(x)) = path_comp_sec(x)
CompTarget_Z(x)[p](id_Rep_Z(x)) = path_comp_func(p)
path_comp_sec(x)[p] = path_comp_func(p)
path_comp_func(p)[z][q] = q ∘ p
```

The two expanded routes that matter most are the primary telescope route:

```
PathInd_transfd(Z)[x][CompMotive_Z(x)](id_Rep_Z(x))[p][z][q]
  = comp_fapp0 Z x y z q p
```

and the derived Sigma-total route:

```
PathInd_funcd(Z)[(x,CompMotive_Z(x))](id_Rep_Z(x))[p][z][q]
  = comp_fapp0 Z x y z q p
```

Appendix C. Diagram Source Notes

All figures in this article are Arrowgram JSON blocks. They are intentionally simple: the diagrams explain object and arrow flow, while the actual normalization claims are made by Lambdapi assertions in `emdash3_2_checks.lp`.